

Assignment /

Assignment #: 21

Topic : Set

Level 1: Easy

1. Union of Sets: Write a Python function `union_sets(set1, set2)` that takes two sets `set1` and `set2` as input and returns their union (all unique elements from both sets).

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	<code>set1 = {1, 2, 3}</code> <code>set2 = {2, 3, 4}</code>	<code>set1 = {'a', 'b', 'c'}</code> <code>set2 = {'b', 'c', 'd'}</code>
Output	<code>{1, 2, 3, 4}</code>	<code>{'a', 'b', 'c', 'd'}</code>

2. Set Difference: Define a function `set_difference(set1, set2)` that returns the elements present in `set1` but not in `set2`.

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	<code>set1 = {1, 2, 3, 4},</code> <code>set2 = {2, 3}</code>	<code>set1 = {'a', 'b', 'c'}</code> <code>set2 = {'b', 'c', 'd'}</code>
Output	<code>{1, 4}</code>	<code>{'a'}</code>

3. Remove Duplicates: Implement a function `remove_duplicates(input_list)` that removes duplicates from a list and returns a set of unique elements.

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	<code>input_list = [1, 2, 2, 3, 4, 4]</code>	<code>input_list = ['a', 'b', 'b', 'c']</code>
Output	<code>{1, 2, 3, 4}</code>	<code>{'a', 'b', 'c'}</code>

Level 2: Medium

1. Count Unique Elements: Implement a function `count_unique_elements(input_list)` that takes a list as input and returns the count of unique elements using a set.

Sample test Cases

Test Cases	Test Case1:	Test Case 2:
Input	<code>input_list = [1, 2, 2, 3, 3, 4]</code>	<code>input_list = ['a', 'b', 'b', 'c']</code>
Output	<code>4</code>	<code>3</code>

2. Set Comprehension: Description: Write a function `set_comprehension(n)` that generates a set of squares of integers from 1 to n using set comprehension.

Sample test Cases

Test Cases	Test Case1:	Test Case 2:
Input	<code>n = 5</code>	<code>n = 3</code>
Output	<code>{1, 4, 9, 16, 25}</code>	<code>{1, 4, 9}</code>

3. Set Intersection Count: Implement a function `set_intersection_count(sets)` that computes the number of elements common to all sets in a list of sets.

Sample test Cases

Test Cases	Test Case1:	Test Case 2:
Input	sets = [{1, 2, 3}, {2, 3, 4}, {3, 4, 5}]	[{'a', 'b'}, {'b', 'c'}, {'c', 'd'}]
Output	1	0

Level 3: Hard

1. Anagram Groups : Implement a function `anagram_groups(words)` that takes a list of words and returns a list of sets where each set contains words that are anagrams of each other.

Sample test Cases

Test Cases	Test Case 1:
Input	words = ['listen', 'silent', 'triangle', 'integral', 'debit card', 'bad credit']
Output	[{'listen', 'silent'}, {'triangle', 'integral'}, {'debit card', 'bad credit'}]

2. Subset Sum to Target: Create a function `subset_sum_to_target(input_set, target_sum)` that finds subsets of `input_set` whose elements sum up to `target_sum`.

Sample test Cases

Test Cases	Test Case1:	Test Case 2:
Input	input_set = {2, 4, 5, 7, 8}, target_sum = 10	input_set = {1, 2, 3, 4, 5} target_sum = 8
Output	[{2, 8}, {4, 6}]	[{3, 5}, {2, 3, 4}, {1, 2, 5}]